Figure A.4, inspired by Jules Blom's component timeline visualization (https://julesblom.com/writing/react-hook-component-timeline), illustrates the component lifecycle phases within the context of functional components, showcasing how these phases are managed using React Hooks.

- *Mounting*—Initial creation and insertion into the DOM, involving a render phase that builds the Virtual DOM and a commit phase that inserts the DOM/refs, sets up layout effects, triggers browser paint, and runs the `useEffect` hook.
- *Updating*—Re-rendering due to state or props changes, involving a render phase that reconciles the Virtual DOM and a commit phase that updates the DOM nodes, cleanups layout effects, unsets DOM refs, sets up DOM refs/layout effects, triggers browser paint, and runs the `useEffect` hook. There is no explicit pre-commit phase. The `useLayoutEffect` hook, which runs synchronously after DOM mutations but before the browser paint, is the closest equivalent to the pre-commit phase functionality (`getSnapshotBeforeUpdate`) in functional components.
- *Unmounting*—Removal from the DOM, skipping the render phase, with the commit phase triggering cleanup of resources, such as subscriptions or timers.

A.6 Handling runtime errors

Error handling is critical for building resilient applications, especially in large-scale, real-world systems where unexpected failures are inevitable. Mastery of patterns like Error Boundaries and global error hooks demonstrates the ability to design applications that gracefully recover from runtime issues and maintain a stable user experience even under failure conditions.

Let’s explore the inner workings of Error Boundaries and global error hooks, highlighting key concepts interviewers often test to assess proficiency in managing complex error scenarios.

A.6.1 Error boundaries